MUSANZE HARVEST INTELLIGENCE

SWE3513 — AI Assignment 2

Student: Cedrick Manzi

Verification code: AI-A2-7196

Dataset: data/AI_A2_2527196.csv

Report date: 17 October 2026

Project Summary

Musanze Harvest Intelligence is a Python machine-learning project that validates agricultural data, trains regression and classification models, performs clustering, saves trained models, and exposes predictions through a FastAPI application.

1. Objectives

The project aims to prepare a farm dataset for analysis, compare candidate machine-learning models, save the selected models, and provide a programmatic interface for predictions. The implementation also includes automated API tests.

2. Dataset and preparation

The generated dataset contains 264 rows before duplicate removal. Validation identified 3 duplicate rows; the cleaned dataset contained 261 rows. Missing values were reported in soil_moisture_pct (13) and fertilizer_kg (13). The preprocessing workflow uses median imputation, scaling, and one-hot encoding for the season feature.

3. Regression results

Linear Regression was selected from the evaluated regression models. On the reported hold-out evaluation, it achieved MAE = 0.88167, RMSE = 1.14908, and R-squared = 0.80443. The evaluated Random Forest regressor achieved MAE = 1.10749, RMSE = 1.33847, and R-squared = 0.73465.

4. Classification results

Logistic Regression was selected for classification. Reported metrics were accuracy = 0.96226, precision = 0.96154, recall = 0.96154, F1-score = 0.96154, and ROC AUC = 0.99430. The Random Forest classifier reported an F1-score of 0.92308. These values reflect the project's evaluation run and should not be interpreted as a guarantee of performance on new data.

5. Clustering results

The clustering workflow compared candidate cluster counts from k=2 through k=6 using silhouette score. The best reported result was k=2 with a silhouette score of approximately 0.15057. This indicates some cluster structure, but the score is modest and the groups should be interpreted cautiously.

6. API and saved models

The FastAPI application provides the root route, /health, /model-info, and /predict. The prediction input includes rainfall, average temperature, soil moisture, fertilizer quantity, field size, season, and cooperative support. Input validation checks constraints such as non-negative rainfall, soil moisture between 0 and 100, positive field size, supported season values A or B, and a binary cooperative-support value. Trained models and the clustering preprocessor are saved under the models directory.

7. Testing

The project's API tests cover the health endpoint, model information, a valid prediction request, and rejection of an invalid season. The latest recorded command was python -m pytest -q, which reported 4 passed and 1 warning. The warning concerned Starlette's test-client integration with httpx; it did not cause the tests to fail.

8. Limitations and future improvements

The models are trained on a generated dataset, so their results may not represent real farm conditions. Future work should evaluate the models on independently collected agricultural data, check for data leakage, use cross-validation, improve clustering interpretation, and monitor model performance after deployment.

9. Conclusion

The project implements an end-to-end machine-learning workflow with data validation, preprocessing, model evaluation, model persistence, a prediction API, and automated tests. Results should be assessed in the context of the generated dataset and the evaluation procedure.